

Java Collections Framework

Detailed Learning Notes • Definitions • Theory • Internal Concepts • Implementation •
Examples • Exercises

Goal: Build a strong understanding of Java Collections from beginner level to interview-ready usage.

Contents

- 1 Introduction to Collections
- 2 Collection vs Collections vs Collection Framework
- 3 Hierarchy and Interfaces
- 4 List – ArrayList, LinkedList, Vector, Stack
- 5 Set – HashSet, LinkedHashSet, TreeSet
- 6 Queue and Deque
- 7 Map – HashMap, LinkedHashMap, TreeMap, Hashtable
- 8 Iterator, ListIterator and for-each
- 9 Comparable and Comparator
- 10 Generics with Collections
- 11 Important Methods
- 12 Time Complexity and Choosing a Collection
- 13 Common Mistakes and Interview Concepts
- 14 Practical Programs
- 15 Exercises with Solutions

1. Introduction to Java Collections

Definition: A collection is an object that represents a group of objects as a single unit. Java Collections Framework (JCF) provides interfaces, classes, and algorithms for storing and manipulating groups of objects.

Before collections, programmers commonly used arrays. Arrays are useful when the size is known and fixed, but real applications frequently need dynamic storage, searching, sorting, insertion, deletion, uniqueness, queues, or key-value relationships. Collections solve these requirements using reusable data structures.

Why do we need Collections?

- Dynamic size: many collection implementations can grow or shrink automatically.
- Ready-made data structures: List, Set, Queue, Deque, and Map implementations are already provided.
- Common operations: add, remove, search, sort, iterate, and check membership.
- Type safety: generics allow collections to store a specified type.
- Better code reuse: developers use tested library implementations instead of implementing every data structure from scratch.

Array vs Collection

Feature	Array	Collection
Size	Fixed after creation	Usually dynamic
Primitive values	Can directly store primitives	Stores objects; primitives use wrapper types via autoboxing
Operations	Mostly manual	Many built-in methods
Data structures	Basic indexed structure	Many specialized structures
Type	Can be homogeneous	Generics provide strong type safety

■ A Collection is not the same thing as the Collection Framework. Collection is an interface; the framework is the larger group of interfaces, classes, and algorithms.

2. Collection vs Collections vs Collection Framework

Collection interface

`java.util.Collection` is the root interface for the main group-oriented data structures such as List, Set, and Queue. It defines common operations like `add()`, `remove()`, `contains()`, `size()`, `isEmpty()`, `clear()`, and `iterator()`.

```
Collection<String> names = new ArrayList<>();
names.add("Amit");
names.add("Ravi");
System.out.println(names.size());
System.out.println(names.contains("Amit"));
```

Collections class

`java.util.Collections` is a utility class containing static methods that operate on collections. Examples include `sort()`, `reverse()`, `shuffle()`, `max()`, `min()`, `frequency()`, `binarySearch()`, and `synchronized` wrappers.

```
List<Integer> nums = new ArrayList<>(Arrays.asList(40, 10, 30, 20));
Collections.sort(nums);
Collections.reverse(nums);
System.out.println(nums);
```

Collection Framework

The Java Collections Framework is the complete architecture: interfaces such as List, Set, Queue, Deque and Map; concrete implementations such as ArrayList, HashSet, HashMap, TreeSet and TreeMap; iterator mechanisms; algorithms; wrappers; and supporting utility classes.

■ Map belongs to the Collections Framework, but Map does not extend Collection because a map stores key-value mappings rather than individual elements.

3. Main Collection Hierarchy

```
Iterable
|
Collection
|-----|-----|
List Set Queue
| | |
ArrayList HashSet PriorityQueue
LinkedList LinkedHashSet Deque
Vector TreeSet |
Stack ArrayDeque

Map (separate from Collection)
|
|-----|-----|-----|
HashMap LinkedHashMap TreeMap Hashtable
```

Core interfaces

List: Ordered collection, allows duplicates, supports index-based access.

Set: Does not permit duplicate elements according to its equality rules.

Queue: Designed for elements waiting to be processed, commonly FIFO, although implementations can use other ordering policies.

Deque: Double-ended queue; insertion and removal can occur at both ends.

Map: Stores key-value mappings. Keys are unique according to the map's equality rules.

Common decision rule

- Need duplicates + insertion order/index? → List.
- Need unique elements? → Set.
- Need processing order / FIFO or priority? → Queue.
- Need both ends? → Deque.
- Need key → value lookup? → Map.

4. List Interface

Definition: List is an ordered collection in which elements have positions (indexes). A List normally permits duplicates and provides methods such as `get(index)`, `set(index, value)`, `add(index, value)`, and `remove(index)`.

4.1 ArrayList

ArrayList is a resizable-array implementation of List. It is usually the default choice when you need fast random/index-based access and frequent reads.

```
import java.util.*;

public class Demo {
    public static void main(String[] args) {
```

```

List<String> names = new ArrayList<>();

names.add("Amit");
names.add("Ravi");
names.add("Amit");

System.out.println(names); // [Amit, Ravi, Amit]
System.out.println(names.get(1)); // Ravi

names.set(1, "Rahul");
names.remove("Amit");

System.out.println(names);
}
}

```

How ArrayList works internally

ArrayList internally uses an array to store elements. When the internal array becomes insufficient, a larger array is allocated and elements are copied into it. This resizing operation is relatively expensive, but it does not happen on every insertion.

Important: ArrayList provides $O(1)$ average indexed access because the element address can be calculated from the index. Insertion/removal in the middle generally requires shifting elements and is $O(n)$.

4.2 LinkedList

LinkedList is a doubly linked list implementation. Each node keeps a reference to the previous node, the element, and the next node. It implements both List and Deque.

```

LinkedList<String> names = new LinkedList<>();

names.add("A");
names.add("B");
names.addFirst("Start");
names.addLast("End");

System.out.println(names);

names.removeFirst();
names.removeLast();

```

LinkedList can be useful when operations at the beginning/end are important and when deque behavior is required. It is generally not the best choice merely because insertion/deletion is needed; access patterns matter.

4.3 Vector and Stack

Vector is a legacy synchronized resizable-array class. Stack extends Vector and provides LIFO operations such as push(), pop(), peek(), and empty(). For modern code, ArrayDeque is generally preferred for stack/deque behavior.

```

Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
System.out.println(stack.pop()); // 20
System.out.println(stack.peek()); // 10

```

5. Set Interface

Definition: Set is a collection that does not allow duplicate elements. The exact ordering behavior depends on the implementation.

5.1 HashSet

HashSet uses hashing and is designed for efficient add, remove, and contains operations on average. It does not guarantee iteration order.

```
Set<String> cities = new HashSet<>();

cities.add("Mumbai");
cities.add("Pune");
cities.add("Mumbai");

System.out.println(cities); // Mumbai appears only once
System.out.println(cities.contains("Pune"));
```

How HashSet decides duplicates

HashSet uses hashCode() to locate a bucket and equals() to determine whether an existing element is equal to the new element. Therefore, when you create custom classes for HashSet, overriding equals() and hashCode() consistently is extremely important.

```
class Employee {
    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Employee)) return false;
        Employee e = (Employee) obj;
        return id == e.id;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(id);
    }
}
```

5.2 LinkedHashSet

LinkedHashSet prevents duplicates like HashSet but maintains insertion order during iteration. It is useful when you need uniqueness plus predictable insertion-order traversal.

5.3 TreeSet

TreeSet stores unique elements in sorted order and is based on a tree structure. Elements must be comparable, or a Comparator must be supplied.

```
Set<Integer> numbers = new TreeSet<>();
numbers.add(40);
numbers.add(10);
numbers.add(30);
numbers.add(10);

System.out.println(numbers); // [10, 30, 40]
```

■ HashSet: uniqueness, no ordering guarantee. LinkedHashSet: uniqueness + insertion order. TreeSet: uniqueness + sorted order.

6. Queue and Deque

Queue: A Queue represents elements waiting to be processed. In a typical FIFO queue, the first element inserted is the first removed.

```
Queue<String> queue = new ArrayDeque<>();

queue.offer("Task-1");
queue.offer("Task-2");
queue.offer("Task-3");

System.out.println(queue.peek()); // Task-1
System.out.println(queue.poll()); // Task-1
System.out.println(queue.poll()); // Task-2
```

offer/add, poll/remove, peek/element

Purpose	Preferred method	Alternative	When empty
Insert	offer(e)	add(e)	offer returns false; add may throw exception
Remove head	poll()	remove()	poll returns null; remove throws exception
Inspect head	peek()	element()	peek returns null; element throws exception

PriorityQueue

PriorityQueue removes elements according to priority rather than insertion order. By default, the smallest element has the highest priority under natural ordering.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(30);
pq.offer(10);
pq.offer(20);

System.out.println(pq.poll()); // 10
System.out.println(pq.poll()); // 20
```

Deque

Deque means double-ended queue. ArrayDeque supports insertion/removal at both ends and can be used as both a queue and a stack.

```
Deque<String> dq = new ArrayDeque<>();

dq.addFirst("B");
dq.addFirst("A");
dq.addLast("C");

System.out.println(dq); // [A, B, C]
System.out.println(dq.removeLast()); // C
```

7. Map Interface

Definition: Map stores associations between keys and values. Each key maps to at most one value. Calling put() with an existing key replaces its associated value.

```
Map<Integer, String> employees = new HashMap<>();

employees.put(101, "Amit");
employees.put(102, "Ravi");
employees.put(101, "Rahul");

System.out.println(employees.get(101)); // Rahul
System.out.println(employees.containsKey(102));
```

7.1 HashMap

HashMap is the most commonly used general-purpose map. It provides average constant-time lookup, insertion, and removal under normal hashing assumptions. It does not guarantee iteration order.

HashMap permits one null key and multiple null values. It is not synchronized.

HashMap internal concept

HashMap uses a hash table. A key's hash is processed to determine a bucket. If multiple keys map to the same bucket, entries are handled using collision-resolution structures. In modern Java implementations, heavily collided buckets can be converted into balanced tree structures under appropriate conditions.

■ Never depend on HashMap iteration order. If order matters, choose an implementation whose contract provides the required order.

7.2 LinkedHashMap

LinkedHashMap maintains a linked ordering of entries. By default it maintains insertion order; it can also be configured for access-order, which is useful for certain cache/LRU-style designs.

7.3 TreeMap

TreeMap keeps keys sorted according to natural ordering or a supplied Comparator. It supports operations such as firstKey(), lastKey(), ceilingKey(), floorKey(), and range-oriented views.

```
Map<Integer, String> marks = new TreeMap<>();
marks.put(30, "C");
marks.put(10, "A");
marks.put(20, "B");

System.out.println(marks); // {10=A, 20=B, 30=C}
```

7.4 Hashtable

Hashtable is a legacy synchronized Map implementation. It does not permit null keys or null values. For most modern applications, HashMap or ConcurrentHashMap is preferred depending on the concurrency requirement.

Map iteration

```
Map<Integer, String> map = new HashMap<>();
map.put(1, "A");
map.put(2, "B");

for (Map.Entry<Integer, String> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}

for (Integer key : map.keySet()) {
    System.out.println(key);
}

for (String value : map.values()) {
    System.out.println(value);
}
```

8. Iteration: for-each, Iterator and ListIterator

The enhanced for loop is simple and readable. Iterator provides controlled traversal and safe removal during iteration. ListIterator is specifically for Lists and supports bidirectional traversal and modification operations.

```
List<String> names = new ArrayList<>(
    Arrays.asList("A", "B", "C", "D")
);

Iterator<String> it = names.iterator();
```

```

while (it.hasNext()) {
    String name = it.next();
    if (name.equals("B")) {
        it.remove();
    }
}

```

```

System.out.println(names);

```

ConcurrentModificationException: A common mistake is structurally modifying a collection directly while an iterator is traversing it. `Iterator.remove()` is designed for the iterator's supported removal operation.

ListIterator

```

List<String> list = new ArrayList<>(
    Arrays.asList("A", "B", "C")
);

ListIterator<String> li = list.listIterator();

while (li.hasNext()) {
    String value = li.next();
    if (value.equals("B")) {
        li.set("X");
    }
}

```

9. Comparable and Comparator

Comparable

`Comparable` defines an object's natural ordering inside the class. The class implements `Comparable` and overrides `compareTo()`.

```

class Student implements Comparable<Student> {
    int marks;

    Student(int marks) {
        this.marks = marks;
    }

    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.marks, other.marks);
    }
}

List<Student> students = new ArrayList<>();
students.add(new Student(80));
students.add(new Student(60));
students.add(new Student(90));

Collections.sort(students);

```

Comparator

`Comparator` defines an external/custom ordering. It is useful when you want multiple different sorting strategies without changing the class's natural ordering.

```

students.sort((a, b) -> Integer.compare(b.marks, a.marks)); // descending

```

Comparable	Comparator
Implemented by the class	Usually supplied separately

Defines natural ordering	Defines custom ordering
Method: compareTo()	Method: compare()
One main natural order is typical	Multiple strategies can coexist

10. Generics with Collections

Generics allow a collection to specify what type of objects it is intended to contain. This provides compile-time type checking and reduces explicit casting.

```
List<String> names = new ArrayList<>();
names.add("Amit");
names.add("Ravi");

// names.add(100); // compile-time error

String first = names.get(0); // no cast required
```

Without generics

```
List list = new ArrayList();
list.add("Amit");
list.add(100);

String s = (String) list.get(1); // ClassCastException at runtime
```

Use parameterized types such as List, Set, Map, and Queue instead of raw collection types.

Diamond operator

```
List<String> names = new ArrayList<>();
```

The <> lets the compiler infer the generic type on the right-hand side.

11. Important Collection Methods

Method	Meaning / Typical use
add(e)	Adds an element
addAll(c)	Adds all elements from another collection
remove(o)	Removes a matching element
contains(o)	Checks whether an element exists
size()	Number of elements
isEmpty()	Checks whether size is zero
clear()	Removes all elements
toArray()	Converts to an array
iterator()	Returns an Iterator
get(index)	List-specific indexed access
set(index,e)	List-specific replacement
put(k,v)	Map insertion/replacement
get(k)	Map lookup
getOrDefault(k,d)	Lookup with fallback
putIfAbsent(k,v)	Insert only if key is absent
remove(k,v)	Remove key only if mapped to value

Method	Meaning / Typical use
entrySet()	Map entries as key-value pairs
keySet()	Map keys
values()	Map values

12. Time Complexity and Choosing the Right Collection

Collection	Get / lookup	Add	Remove	Ordering
ArrayList	O(1) by index	O(1) amortized at end	O(n) middle	Insertion order
LinkedList	O(n) by index	O(1) at known end/node	O(1) at known end/node	Insertion order
HashSet	O(1) average	O(1) average	O(1) average	No guarantee
LinkedHashSet	O(1) average	O(1) average	O(1) average	Insertion order
TreeSet	O(log n)	O(log n)	O(log n)	Sorted
HashMap	O(1) average	O(1) average	O(1) average	No guarantee
LinkedHashMap	O(1) average	O(1) average	O(1) average	Insertion/access order
TreeMap	O(log n)	O(log n)	O(log n)	Sorted keys
PriorityQueue	peek O(1)	O(log n)	poll O(log n)	Priority order

■ Complexity figures are typical/average-case descriptions and depend on the implementation and operation. Hash-based structures can degrade under collisions; resizing and other details can affect individual operations.

13. Important Concepts and Common Interview Questions

Why does HashSet not allow duplicates?

Because it uses hashing plus equality checks. If an equal element already exists according to the set's rules, the new element is not added.

Why must equals() and hashCode() agree?

If two objects are equal according to equals(), they must return the same hashCode(). Otherwise hash-based collections can place logically equal objects in different buckets and fail to behave as expected.

HashMap vs Hashtable

- HashMap is modern and generally preferred for non-concurrent use.
- Hashtable is legacy and synchronized.
- HashMap allows null key/value entries; Hashtable does not allow null keys or values.
- For high-concurrency map access, ConcurrentHashMap is usually considered instead of Hashtable.

ArrayList vs LinkedList

ArrayList is generally better for frequent indexed reads and append-heavy workloads. LinkedList provides efficient operations at known ends and implements Deque, but indexed access is linear and each node has extra memory overhead.

HashMap vs TreeMap

HashMap is intended for fast average lookup without sorted keys. TreeMap maintains keys in sorted order and provides logarithmic-time tree operations.

Fail-fast behavior

Many standard collection iterators are fail-fast in the sense that they may detect structural modification outside the iterator and throw `ConcurrentModificationException`. This is a bug-detection mechanism, not a synchronization guarantee.

14. Practical Programs

Program 1 – Remove duplicates while preserving insertion order

```
import java.util.*;

public class RemoveDuplicates {
    public static void main(String[] args) {
        List<Integer> numbers =
            Arrays.asList(10, 20, 10, 30, 20, 40);

        Set<Integer> unique = new LinkedHashSet<>(numbers);

        System.out.println(unique);
    }
}
```

Program 2 – Count frequency using HashMap

```
import java.util.*;

public class Frequency {
    public static void main(String[] args) {
        String text = "java is easy java is powerful";

        Map<String, Integer> freq = new HashMap<>();

        for (String word : text.split(" ")) {
            freq.put(word, freq.getOrDefault(word, 0) + 1);
        }

        System.out.println(freq);
    }
}
```

Program 3 – Sort employees by salary

```
import java.util.*;

class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public String toString() {
        return name + " : " + salary;
    }
}

public class SortEmployees {
    public static void main(String[] args) {
        List<Employee> list = new ArrayList<>();

        list.add(new Employee("Amit", 50000));
        list.add(new Employee("Ravi", 70000));
        list.add(new Employee("Neha", 60000));
    }
}
```

```
list.sort(Comparator.comparingDouble(e -> e.salary));

System.out.println(list);
}
}
```

Program 4 – Simple employee lookup

```
Map<Integer, Employee> employees = new HashMap<>();

employees.put(101, new Employee("Amit", 50000));
employees.put(102, new Employee("Ravi", 70000));

Employee e = employees.get(102);

if (e != null) {
    System.out.println(e);
}
```

15. Exercises

Beginner

- Create an ArrayList, add 10 numbers, and print them using a for-each loop.
- Create a List containing duplicate city names. Remove duplicates using a Set.
- Create a HashSet and check whether a given number exists.
- Create a HashMap for employee ID and employee name. Perform insert, search, update, and delete.
- Create a Queue for customer names. Add five customers and process them one by one.

Intermediate

- Count the frequency of every character in a String using HashMap.
- Find the first non-repeated character in a string using a LinkedHashMap.
- Sort a list of integers in ascending and descending order.
- Create an Employee class and sort employees by name and then by salary using Comparator.
- Remove all even numbers from an ArrayList using Iterator.
- Use TreeSet to print unique numbers in sorted order.
- Use TreeMap to store student roll numbers and marks and print the entries in sorted roll-number order.

Advanced

- Build a small employee-management program using Map with add, search, update, delete, and display operations.
- Implement an LRU-style cache concept using LinkedHashMap.
- Given a list of words, group words by their length using Map>.
- Find the top three largest numbers using PriorityQueue.
- Create a custom class for HashSet and correctly implement equals() and hashCode().
- Design a program that stores products by category using Map>.

16. Exercise Solutions – Selected

Solution: Character frequency

```
import java.util.*;

public class CharFrequency {
    public static void main(String[] args) {
        String text = "banana";

        Map<Character, Integer> freq = new HashMap<>();

        for (char ch : text.toCharArray()) {
            freq.put(ch, freq.getOrDefault(ch, 0) + 1);
        }

        System.out.println(freq);
    }
}
```

Solution: Remove even numbers safely

```
List<Integer> numbers =
    new ArrayList<>(Arrays.asList(10, 15, 20, 25, 30));

Iterator<Integer> it = numbers.iterator();

while (it.hasNext()) {
    if (it.next() % 2 == 0) {
        it.remove();
    }
}

System.out.println(numbers);
```

Solution: Sort objects using Comparator

```
employees.sort(
    Comparator.comparingDouble((Employee e) -> e.salary)
        .thenComparing(e -> e.name)
);
```

17. Quick Revision Cheat Sheet

Requirement	Recommended starting point
Duplicates + index	ArrayList
Unique values	HashSet
Unique + insertion order	LinkedHashSet
Unique + sorted	TreeSet
FIFO processing	ArrayDeque / Queue
Stack behavior	ArrayDeque
Priority processing	PriorityQueue
Key-value lookup	HashMap
Key-value + insertion order	LinkedHashMap
Key-value + sorted keys	TreeMap
Custom sorting	Comparator
Natural sorting	Comparable
Safe iterator removal	Iterator.remove()

Suggested learning order: Collection basics → List → Set → Queue/Deque → Map → Iterator → Comparable/Comparator → Generics → complexity → practical programs. After completing the exercises, implement one small Employee Management project using ArrayList, HashSet, HashMap, Comparator, and Iterator.